

```
!pip install pyspark
```

Requirement already satisfied: pyspark in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/pytho

```
from google.colab import files
```

```
uploaded = files.upload()
```

archive.zip

archive.zip(application/x-zip-compressed) - 45212625 bytes, last modified: 7/23/2026 - 100% done

Saving archive.zip to archive.zip

```
import zipfile
```

```
import os
```

```
# Assuming the uploaded file is named 'archive.zip'
```

```
zip_file_path = 'archive.zip'
```

```
# Check if the zip file exists
```

```
if os.path.exists(zip_file_path):
```

```
    with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
```

```
        zip_ref.extractall('.') # Extract all contents to the current directory
```

```
        print(f"Successfully unzipped '{zip_file_path}'")
```

```
        print("Extracted files:")
```

```
        for filename in zip_ref.namelist():
```

```
            print(f"- {filename}")
```

```
else:
```

```
    print(f"Error: Zip file '{zip_file_path}' not found.")
```

Successfully unzipped 'archive.zip'

Extracted files:

- faker_employee.csv

```
# Small lookup table: department budgets
```

```
dept_data = [
```

```
    ("Sales", 500000),
```

```
    ("Legal", 300000),
```

```
    ("R&D", 800000),
```

```
    ("Training and Development", 200000)
```

```
]
```

```
dept_df = spark.createDataFrame(dept_data, ["Department", "Budget"])
```

```
# RDD version of the same lookup, as (key, value) pairs
```

```
dept_rdd = sc.parallelize(dept_data).map(lambda x: (x[0], x[1]))
```

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum as spark_sum, rank
from pyspark.sql.window import Window
import csv
from io import StringIO

spark = SparkSession.builder \
    .appName("RDD_Practice") \
    .master("local[*]") \
    .getOrCreate()

sc = spark.sparkContext
rdd = sc.textFile("/content/faker_employee.csv")

header = rdd.first()
data = rdd.filter(lambda row: row != header)

def parse_csv_line(line):
    return next(csv.reader(StringIO(line)))

employee_rdd = data.map(parse_csv_line)

# guard against malformed/short rows before doing anything with indexes
clean_rdd = employee_rdd.filter(lambda row: len(row) >= 8 and row[3].strip() !=

print("Total rows:", employee_rdd.count())
print("Clean rows:", clean_rdd.count())
clean_rdd.take(3)

```

```

Total rows: 2000000
Clean rows: 1000000
[['1',
  'Nicholas',
  'Martin',
  'Sales',
  '50819',
  '2022-11-16',
  'dannymays@example.com',
  '14632 Ashley Trafficway Suite 345'],
 ['2',
  'Joy',
  'Miller',
  'Legal',
  '58024',
  '2023-02-03',
  'heather59@example.net',
  '00670 Oliver Harbors'],
 ['3',
  'Thomas',
  'Roth',
  'R&D',
  '54572',
  '2021-08-18',

```

```
'garyhogan@example.com',  
'3024 John Turnpike']]
```

```
emp_pair_rdd = clean_rdd.map(lambda row: (row[3], row)) # (Department, row)  
joined_rdd = emp_pair_rdd.join(dept_rdd) # (Department, (row, b  
joined_rdd.take(3)
```

```
[('Sales',  
  (['1',  
    'Nicholas',  
    'Martin',  
    'Sales',  
    '50819',  
    '2022-11-16',  
    'dannymays@example.com',  
    '14632 Ashley Trafficway Suite 345'],  
    500000)),  
 ('Sales',  
  (['4',  
    'Brittney',  
    'Morgan',  
    'Sales',  
    '72174',  
    '2022-10-03',  
    'wcervantes@example.org',  
    '374 Cook Dam Suite 080'],  
    500000)),  
 ('Sales',  
  (['22',  
    'Angela',  
    'Jenkins',  
    'Sales',  
    '61763',  
    '2022-07-16',  
    'andersontyrone@example.net',  
    '0350 Black Lights'],  
    500000))]
```

Start coding or [generate](#) with AI.

